

Tema 2 — Overfitting, Underfitting, Bias/Varianza, y Métricas de Evaluación (Precision, Recall, F1, Accuracy)

0. GLOSARIO RÁPIDO DE MÉTRICAS

- **Accuracy (Exactitud):** proporción de predicciones totales (positivas y negativas) que el modelo acertó. $\text{Accuracy} = (TP+TN)/(TP+TN+FP+FN)$
 - **Precision (Precisión):** de todo lo que el modelo dijo que era positivo, qué proporción realmente lo era. $\text{Precision} = TP/(TP+FP)$
 - **Recall (Sensibilidad):** de todo lo que realmente era positivo, qué proporción el modelo logró detectar. $\text{Recall} = TP/(TP+FN)$
 - **F1-Score:** media armónica entre Precision y Recall, balancea ambas en un solo número. $\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$
-

PARTE 1 — OVERFITTING, UNDERFITTING, BIAS Y VARIANZA

1. ¿QUÉ ES OVERFITTING (SOBREAJUSTE)?

Ocurre cuando un modelo se ajusta demasiado a los datos de entrenamiento, capturando tanto los patrones reales como el ruido y los detalles irrelevantes.

Es como "memorizar" los ejemplos del parcial en vez de aprender los conceptos: te va perfecto en esos ejercicios, pero te va mal en cualquier ejercicio nuevo.

Características:

- El modelo tiene un desempeño EXCELENTE en entrenamiento
- El modelo tiene un desempeño MALO en test/datos nuevos
- No generalizó: aprendió los datos de entrenamiento de memoria

¿Cuándo ocurre?

- Modelos demasiado complejos (árboles muy profundos, redes con muchos parámetros)
- Poco volumen de datos de entrenamiento
- Entrenar demasiadas épocas (en redes neuronales)
- No usar regularización
- Alta varianza del modelo

¿Cómo se detecta?

Comparando las métricas de entrenamiento vs las de test:

```
Entrenamiento: F1 = 0.98    <    muy alto
Test:           F1 = 0.55    <    mucho más bajo
                                   → HAY OVERFITTING
```

La brecha grande entre la performance de train y test es la señal.

2. ¿QUÉ ES UNDERFITTING (SUBAJUSTE)?

Ocurre cuando un modelo es demasiado simple para capturar la relación subyacente en los datos.

Es como estudiar solo el título de cada tema: no aprendés lo suficiente para resolver nada, ni siquiera los ejercicios que ya viste.

Características:

- El modelo tiene un desempeño MALO en entrenamiento
- El modelo tiene un desempeño MALO en test
- No aprendió nada: ni los datos de entrenamiento pudo captar

¿Cuándo ocurre?

- Modelos demasiado simples (regresión lineal para un problema no lineal)
- Pocos parámetros / pocos features
- Regularización excesiva (se penalizan tanto los pesos que el modelo no puede aprender)
- Dataset pobre o insuficiente
- Alto sesgo (bias) del modelo

¿Cómo se detecta?

Comparando las métricas de entrenamiento vs test:

```
Entrenamiento: F1 = 0.25    <    malo
Test:           F1 = 0.30    <    también malo
                                   → HAY UNDERFITTING
```

Ambas métricas son malas. No es que una sea buena y la otra mala (eso sería overfitting), sino que las dos son malas.

3. CUADRO COMPARATIVO

	Overfitting	Underfitting
Performance en train	Alta (muy buena)	Baja (mala)
Performance en test	Baja (mala)	Baja (mala)
Problema	Memorizó, no generalizó	No aprendió nada
Causa típica	Modelo muy complejo	Modelo muy simple
Varianza	Alta	Baja
Sesgo (Bias)	Bajo	Alto
Solución	Simplificar el modelo, regularizar	Complejizar el modelo, más features

4. BIAS (SESGO) Y VARIANCE (VARIANZA)

Estos dos conceptos están directamente relacionados con overfitting y underfitting. Entenderlos es clave porque los preguntan juntos.

Bias (Error de sesgo)

Es la diferencia entre el valor esperado del estimador (la predicción promedio del modelo) y el valor real.

- **Bias alto** = el modelo es muy simple, no se ajustó a los datos de entrenamiento → produce errores altos en TODAS las muestras (entrenamiento, validación y test) → **underfitting**
- **Bias bajo** = el modelo captura bien los patrones de los datos

Ejemplo: si intentás predecir una curva con una recta, la recta nunca va a captar la curvatura → sesgo alto.

Variance (Varianza)

Es cuánto varía la predicción del modelo cuando cambiás los datos de entrenamiento.

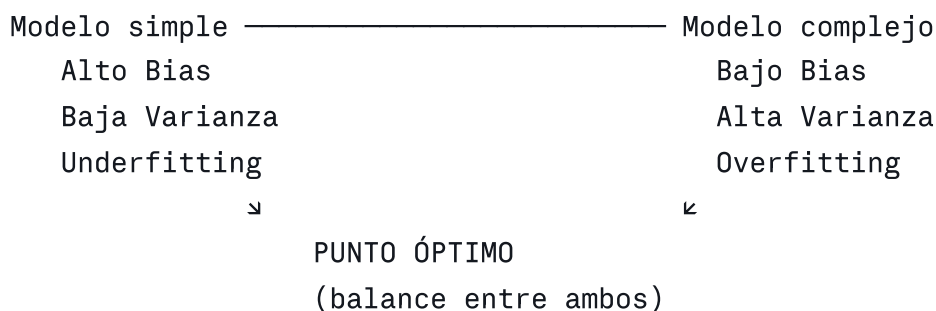
- **Varianza baja** = cambiar los datos de entrenamiento produce cambios pequeños en la predicción → modelo estable
- **Varianza alta** = pequeños cambios en el dataset de entrenamiento producen grandes cambios en la predicción → el modelo se ajustó demasiado a los datos específicos de entrenamiento → **overfitting**

Ejemplo: un árbol de decisión muy profundo cambia completamente si le sacás unos pocos datos de entrenamiento → varianza alta.

El Trade-off Bias-Varianza

No se pueden minimizar ambos al mismo tiempo. Hay un punto óptimo:

Error total = $\text{Bias}^2 + \text{Varianza} + \text{Ruido irreducible}$



- Si hacés el modelo más complejo: baja el bias pero sube la varianza
- Si hacés el modelo más simple: baja la varianza pero sube el bias
- El objetivo es encontrar el punto donde el error total es mínimo

5. ¿CÓMO EVITAR EL OVERFITTING?

5.1. Reducir la complejidad del modelo

- Usar menos parámetros / capas / neuronas
- Usar árboles menos profundos (controlar max_depth)
- Elegir un modelo más simple (ej: regresión lineal en vez de red neuronal)

5.2. Regularización L1 y L2

- **L1 (Lasso):** Agrega $\lambda * \sum |w|$ a la función de pérdida. Lleva algunos pesos a exactamente 0, eliminando features irrelevantes.
- **L2 (Ridge):** Agrega $\lambda * \sum w^2$ a la función de pérdida. Hace los pesos pequeños pero no cero, reduciendo la influencia de features dominantes.
- Ambas penalizan que los pesos crezcan demasiado, forzando al modelo a ser más "conservador".

5.3. Dropout (solo para redes neuronales)

- "Apaga" un porcentaje de neuronas al azar en cada iteración de entrenamiento
- Fuerza a la red a no depender de neuronas específicas
- En test/producción se usan TODAS las neuronas

5.4. Early Stopping (solo para redes neuronales)

- Monitorear el error del conjunto de validación en cada epoch
- Si el error de validación empieza a SUBIR → parar el entrenamiento
- Quedarse con los pesos de la iteración donde el error de validación era mínimo
- Evita que la red siga memorizando los datos de entrenamiento

5.5. Data Augmentation (Aumento de datos)

- Generar datos nuevos a partir de los existentes con transformaciones (rotaciones, recortes, zoom, inversiones, cambios de brillo, etc.)
- Aumenta la diversidad del dataset sin recolectar datos nuevos
- Muy común en problemas de imágenes (visión por computadora)
- Mejora la generalización

5.6. Validación cruzada (K-Fold Cross Validation)

- Dividir los datos de entrenamiento en K subconjuntos (folds)
- Entrenar K veces, cada vez usando K-1 folds para entrenar y 1 para validar
- Permite evaluar la performance real del modelo y optimizar hiperparámetros (con GridSearchCV) para encontrar la configuración que generalice mejor

5.7. Poda de árboles (solo para árboles de decisión)

- En árboles C4.5: después de construir el árbol, se eliminan nodos cuya eliminación NO aumenta el error en el conjunto de test
- Controlar hiperparámetros como max_depth, min_samples_split, etc.
- Un árbol muy profundo memoriza ruido → overfitting
- Un árbol muy poco profundo no captura relaciones → underfitting

5.8. Más datos de entrenamiento

- Cuantos más datos diversos tenga el modelo, más difícil es que memorice patrones espurios
- Combinable con Data Augmentation

6. ¿CÓMO EVITAR EL UNDERFITTING?

- **Aumentar la complejidad del modelo:** más capas, más neuronas, más parámetros, árboles más profundos
- **Reducir la regularización:** si la penalización L1/L2 es muy alta, el modelo no puede aprender

- **Agregar más features:** Feature Engineering, crear variables nuevas
 - **Entrenar más tiempo:** más epochs (en redes neuronales)
 - **Mejorar el dataset:** más datos, datos más representativos
-

7. RELACIÓN CON OTROS TEMAS DE LA MATERIA

Con árboles de decisión

- Árbol muy profundo (sin poda) → overfitting
- La poda en C4.5 es explícitamente un método para evitar overfitting
- max_depth controla la complejidad: muy alto → overfitting, muy bajo → underfitting

Con ensambles

- **Bagging (Random Forest):** Reduce la VARIANZA → ayuda contra overfitting
- **Boosting (AdaBoost, GradientBoost):** Reduce el SESGO → ayuda contra underfitting
- Los ensambles en general logran un mejor trade-off bias-varianza que los modelos individuales
- Cuidado: cascadas muy profundas pueden causar overfitting

Con redes neuronales

- Los 4 métodos de regularización (L1/L2, Dropout, Early Stopping, Data Augmentation) son todas técnicas para evitar overfitting
- Más capas/neuronas = más capacidad = más riesgo de overfitting
- Los optimizadores (Adam, SGD, etc.) no son métodos de regularización en sí mismos, pero la elección del learning rate afecta: muy alto puede no converger, muy bajo puede overfittear

Con train/test split

- Separar datos en train y test es FUNDAMENTAL para detectar overfitting
- Si solo evaluas en train, nunca sabés si el modelo generalizó
- Data leakage (fuga de datos): si usás información del test durante el preprocesamiento (por ejemplo, normalizar con la media del test), estás "contaminando" la evaluación y no detectás overfitting real

Con reducción de dimensionalidad

- Reducir features puede ayudar a evitar overfitting (menos parámetros = modelo más simple)
- PCA elimina las dimensiones que menos varianza explican

Con hiperparámetros

- Los hiperparámetros controlan el balance entre sesgo y varianza
 - Se optimizan con GridSearchCV / RandomSearchCV usando validación cruzada
 - Definen la complejidad del modelo
-

8. CASO PRÁCTICO: INTERPRETAR MÉTRICAS

Este tipo de ejercicio aparece en los exámenes. Te dan métricas de train y test y tenés que diagnosticar.

Caso A

F1 en entrenamiento: 0.95
F1 en test: 0.50

Diagnóstico: OVERFITTING. El modelo tiene excelente performance en entrenamiento pero mala en test. Memorizó los datos de entrenamiento pero no generalizó. Solución: regularizar, simplificar el modelo, más datos.

Caso B

F1 en entrenamiento: 0.25
F1 en test: 0.40

Diagnóstico: UNDERFITTING. El modelo tiene mala performance tanto en entrenamiento como en test. No aprendió los patrones. Solución: modelo más complejo, más features, menos regularización.

Ojo con la trampa: en este caso el test es MEJOR que el train. ¿Significa que está overfitteando al revés? No. Simplemente el modelo es tan malo que no aprendió nada, y las diferencias son ruido estadístico. Lo que importa es que AMBAS métricas son malas → underfitting.

Caso C

F1 en entrenamiento: 0.85
F1 en test: 0.82

Diagnóstico: BUEN MODELO. La diferencia entre train y test es pequeña (3%), y ambas métricas son razonablemente altas. El modelo generalizó bien.

Caso D

Accuracy en entrenamiento: 0.99

Accuracy en test: 0.98

Diagnóstico: Podría ser un buen modelo O podría haber un problema de clases desbalanceadas. Si el 98% de los datos son de una clase, un modelo que prediga siempre esa clase tendría 98% de accuracy sin haber aprendido nada. Por eso Accuracy sola no alcanza → hay que mirar también Precision, Recall, F1.

PREGUNTAS DE PRÁCTICA — PARTE 1 (Overfitting/Underfitting)

Pregunta 1 (Desarrollo — aparece textual en el ejemplo de parcial)

Explique brevemente los conceptos de Overfitting y Underfitting. Mencione cómo puede detectarse cada una de estas situaciones y qué se podría hacer para evitarlas.

Pregunta 2 (V/F — aparece en el ejemplo de parcial)

Se entrena un modelo predictivo y se evalúa con F1-Score:

- F1-Score en entrenamiento: 0.25
- F1-Score en test: 0.4

Indique V o F:

- a) El modelo generaliza muy bien para datos nuevos.
- b) El modelo podría estar sobreajustando a los datos de entrenamiento (overfitting).
- c) Hay un problema en los datos de test, se debería evaluar en otro conjunto.
- d) El modelo podría estar subajustando a los datos de entrenamiento (underfitting).

Pregunta 3 (Desarrollo — de los simulacros del Notion)

Explique los siguientes conceptos y cómo se relacionan con la complejidad de un modelo de aprendizaje automático: a) Error de Sesgo (Bias). b) Overfitting (sobreajuste).

Pregunta 4 (V/F)

- a) La técnica de poda se utiliza en los árboles C4.5 para evitar el underfitting.
- b) Dropout y Early Stopping son métodos de regularización que ayudan a evitar el overfitting en redes neuronales.
- c) Un modelo con alta varianza es un modelo que produce errores similares en todos los conjuntos de datos.
- d) Bagging reduce la varianza de la clasificación.

Pregunta 5 (Desarrollo)

¿Qué relación tiene la complejidad de un modelo con el overfitting y el underfitting? ¿Cómo se relaciona esto con el trade-off bias-varianza?

Pregunta 6 (Análisis de caso)

Se entrenaron dos modelos de clasificación y se obtuvieron estos resultados:

Modelo A:

- Accuracy en train: 0.97
- Accuracy en test: 0.60

Modelo B:

- Accuracy en train: 0.78
- Accuracy en test: 0.75

¿Qué modelo elegiría y por qué? ¿Qué problema tiene el otro modelo?

RESPUESTAS MODELO — PARTE 1

Respuesta 1

Overfitting: Significa que el modelo sobre-ajustó los datos de entrenamiento. Se detecta cuando el modelo obtiene muy buenas métricas en entrenamiento pero al evaluar con el conjunto de prueba estas bajan significativamente, lo que indica que el modelo no generalizó. Puede deberse a una alta varianza o a un modelo muy complejo. Para evitarlo se puede usar validación cruzada para optimizar hiperparámetros, aplicar regularización (L1, L2, Dropout, Early Stopping), usar Data Augmentation o intentar con un modelo más simple.

Underfitting: Significa que el modelo no fue capaz de encontrar patrones en los datos de entrenamiento. Se detecta cuando el modelo tiene mala performance tanto en entrenamiento como en test. Puede deberse a que el modelo es muy simple para la complejidad del problema, a un alto sesgo (bias), regularizaciones muy estrictas o un dataset pobre. Para evitarlo se puede usar un modelo más complejo, reducir las regularizaciones o mejorar el dataset.

Respuesta 2

a) **FALSO.** Un F1 de 0.4 en test no es una buena generalización. Además el modelo tampoco aprendió bien los datos de entrenamiento ($F1 = 0.25$).

b) **FALSO.** Para que haya overfitting, el modelo debería tener buena performance en entrenamiento y mala en test. Acá la performance en entrenamiento (0.25) es peor que en test (0.4), lo cual no es indicativo de sobreajuste.

c) **FALSO.** No hay evidencia de un problema específico en los datos de test. El problema está en el modelo, que no logra aprender los patrones.

d) **VERDADERO.** El modelo tiene mala performance tanto en entrenamiento (0.25) como en test (0.4), lo que indica que no logró capturar los patrones subyacentes → underfitting.

Respuesta 3

a) **Error de Sesgo (Bias):** Es la diferencia entre el valor esperado de la predicción del modelo y el valor real. Cuando el sesgo es muy alto, quiere decir que el modelo es muy simple y no fue capaz de ajustarse a los datos de entrenamiento, lo que suele producir underfitting. A medida que aumentamos la complejidad del modelo, el bias tiende a disminuir.

b) **Overfitting (sobreajuste):** Ocurre cuando el modelo sobre-ajustó los datos de entrenamiento, aprendiéndolos de memoria incluyendo el ruido. Al evaluar con datos nuevos, las métricas son malas, indicando que no generalizó. Esto se relaciona con una alta varianza: el modelo es tan complejo que es muy sensible a los datos específicos de entrenamiento. A medida que aumentamos la complejidad del modelo, aumenta el riesgo de overfitting.

Respuesta 4

a) **FALSO.** La poda en C4.5 se utiliza para evitar el OVERFITTING, no el underfitting. La poda reduce la complejidad del árbol eliminando nodos que no mejoran la clasificación, evitando que el árbol memorice ruido.

b) **VERDADERO.** Dropout apaga neuronas al azar durante el entrenamiento, y Early Stopping detiene el entrenamiento cuando el error de validación empieza a subir. Ambos son métodos de regularización contra el overfitting.

c) **FALSO.** Un modelo con alta varianza es sensible a cambios en los datos de entrenamiento: pequeños cambios producen grandes diferencias en las predicciones. Esto suele asociarse a overfitting, no a errores similares en todos los conjuntos.

d) **VERDADERO.** Bagging (y Random Forest) entrena múltiples modelos sobre muestras bootstrap y combina sus predicciones. Esto reduce la varianza de la clasificación al promediar las predicciones de múltiples modelos.

Respuesta 5

Un modelo simple (pocos parámetros, pocas capas, árbol poco profundo) tiene alto sesgo y baja varianza: no logra capturar la complejidad del problema (underfitting), pero sus predicciones son estables ante cambios en los datos.

Un modelo complejo (muchos parámetros, muchas capas, árbol muy profundo) tiene bajo sesgo y alta varianza: captura bien los patrones pero también el ruido (overfitting), y sus predicciones cambian mucho con datos diferentes.

El trade-off bias-varianza dice que el error total del modelo es la suma de $\text{bias}^2 + \text{varianza} + \text{ruido irreducible}$. Al aumentar la complejidad, el bias baja pero la varianza sube. Al disminuirla, la varianza baja pero el bias sube. El objetivo es encontrar el punto intermedio donde el error total es mínimo, logrando un buen balance entre ajuste y generalización.

Respuesta 6

Elegiría el **Modelo B** porque tiene una diferencia pequeña entre train (0.78) y test (0.75), lo que indica que generalizó bien. Si bien su accuracy no es perfecta, es consistente.

El **Modelo A** tiene un problema de overfitting: tiene 0.97 en train pero solo 0.60 en test, una caída de 37 puntos. Memorizó los datos de entrenamiento pero no generalizó a datos nuevos. Se podría intentar regularizarlo, simplificar su complejidad, o usar validación cruzada para optimizar hiperparámetros.

PARTE 2 — PRECISION, RECALL, F1-SCORE Y MATRIZ DE CONFUSIÓN

Cuarto tema más frecuente (6/9 exámenes). Casi siempre viene como ejercicio de CÁLCULO (te dan una matriz de confusión o valores de TP/TN/FP/FN y tenés que calcular métricas), aunque también aparece como pregunta conceptual de "cuándo usar cada una".

9. ¿PARA QUÉ SIRVEN LAS MÉTRICAS?

Las métricas sirven para poder **comparar entre modelos** de un mismo tipo. Es una forma de medir cuál modelo funciona mejor. La métrica que conviene usar depende de la data disponible y del tipo de problema:

- **Clasificación** → Precision, Recall, Accuracy, F1-Score, Matriz de Confusión
- **Regresión** → MAE, MSE, RMSE, R^2

Un problema de clasificación implica que el modelo predice una categoría y que los datos ya están pre-categorizados (labeled dataset).

10. LOS 4 VALORES BÁSICOS: TP, TN, FP, FN

Todo parte de comparar lo que el modelo predijo contra lo que realmente era verdad.

	Predicción Positiva	Predicción Negativa
Real Positivo	TP (True Positive)	FN (False Negative)
Real Negativo	FP (False Positive)	TN (True Negative)

- **TP (Verdadero Positivo):** el modelo predijo positivo y ACERTÓ (era positivo de verdad)
- **TN (Verdadero Negativo):** el modelo predijo negativo y ACERTÓ (era negativo de verdad)
- **FP (Falso Positivo):** el modelo predijo positivo pero SE EQUIVOCÓ (en realidad era negativo) — también llamado "error de tipo I"
- **FN (Falso Negativo):** el modelo predijo negativo pero SE EQUIVOCÓ (en realidad era positivo) — también llamado "error de tipo II"

Truco para no confundirse: la SEGUNDA palabra (Positive/Negative) es lo que el modelo PREDIJO. La PRIMERA palabra (True/False) indica si acertó o no con esa predicción.

11. MATRIZ DE CONFUSIÓN

Es una tabla que da un "pantallazo" de cómo clasifica el modelo.

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Se busca que la diagonal (TP y TN) sea lo más alta posible — eso significa que el modelo acierta más seguido cuando clasifica.

Ejemplo numérico (de las diapos, "modelo bueno" vs "modelo con fallas")

Modelo Bueno:				Modelo con Fallas:			
		Predicted				Predicted	
		Pos	Neg			Pos	Neg
Pos		23	7	Pos		23	7
Neg		10	60	Neg		58	12

Ambos modelos tienen los mismos TP y FN (23 y 7), pero el "modelo con fallas" tiene muchos más FP (58 vs 10) — clasifica como positivo muchos casos que en realidad son negativos.

Matriz de Confusión N-aria (para más de 2 clases)

Se usa cuando la clasificación no es binaria. Es la misma idea: la diagonal tiene la cantidad de aciertos, y los demás cuadros son predicciones erradas (fila = clase real, columna = clase predicha).

		Predicted					
		A	B	C	D	E	F
A	[	13	1	1	0	2	0]
B	[	3	9	6	0	1	0]
C	[	0	0	16	2	0	0]
D	[	0	0	0	13	0	0]
E	[	0	0	0	0	15	0]
F	[	0	0	1	0	0	15]

12. LAS FÓRMULAS PRINCIPALES

Precision (Precisión)

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Responde: **de todo lo que el modelo dijo que era positivo, ¿qué proporción realmente lo era?** Mide qué tan "confiable" es una predicción positiva del modelo.

Recall (Sensibilidad / Exhaustividad)

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Responde: **de todos los casos que REALMENTE eran positivos, ¿qué proporción el modelo logró detectar?** Mide qué tan bien el modelo "atrapa" todos los positivos reales, sin dejar escapar casos.

Accuracy (Exactitud)

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Responde: **de todas las predicciones (positivas y negativas), ¿qué proporción total acertó?** Es la métrica más simple e intuitiva, pero tiene un problema grave con clases desbalanceadas (ver sección 6).

F1-Score

$$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Es la media armónica entre Precision y Recall. Sirve para tener una única métrica que balancea ambas, especialmente útil cuando querés que ninguna de las dos sea muy baja.

¿Por qué media armónica y no promedio simple? Porque la media armónica castiga fuerte cuando una de las dos métricas es muy baja. Si Precision=1.0 y Recall=0.0, un promedio

simple daría 0.5 (parece razonable), pero la media armónica da 0 (correctamente refleja que el modelo es inútil, porque no detecta nada).

13. EJEMPLO COMPLETO RESUELTO (con el ejemplo de Argentina de las diapos)

Un modelo predice si Argentina va a ganar sus próximos 10 partidos. El objetivo declarado es: *"No me importa ganar cada juego que juega la selección, sino que los que yo apueste, gane."* (esto importa para saber qué métrica priorizar — ver sección 6)

Resultado de las 10 predicciones:

TP = 8 (predijo que ganaba, y ganó)
TN = 0 (predijo que perdía, y perdió) — no hubo ninguna
FN = 0 (predijo que perdía, pero ganó) — no hubo ninguna
FP = 2 (predijo que ganaba, pero no ganó)

Cálculos:

Precision = $TP / (TP+FP) = 8 / (8+2) = 8/10 = 0,80$
Recall = $TP / (TP+FN) = 8 / (8+0) = 8/8 = 1,0$
F1-Score = $2 * (0,80 * 1,0) / (0,80 + 1,0) = 2 * 0,8 / 1,8 = 0,889$

Interpretación: El modelo acertó el 80% de las veces que apostó (Precision=0,80). Además, atrapó el 100% de los partidos que Argentina efectivamente ganó (Recall=1,0) — no se le escapó ningún partido ganado sin haberlo predicho.

14. ¿CUÁNDO PRIORIZAR PRECISION Y CUÁNDO RECALL?

Esto es EXTREMADAMENTE preguntado — depende del costo relativo de cada tipo de error (FP vs FN) en el problema específico.

Priorizar PRECISION (minimizar Falsos Positivos)

Cuando un Falso Positivo es muy costoso. Ejemplos:

- **Filtro de spam:** un email importante (no-spam) marcado como spam (FP) puede hacer que pierdas un mensaje crítico. Preferís que se cuele algún spam antes que perder un email real.
- **El ejemplo de Argentina:** "no me importa ganar cada juego, sino que los que aposté, gane" → si predigo que gana y no gana, pierdo plata (FP costoso). No me importa tanto perderme de apostar en partidos que sí ganó (FN) porque ahí simplemente no aposté, no perdí nada.

Priorizar RECALL (minimizar Falsos Negativos)

Cuando un Falso Negativo es muy costoso. Ejemplos:

- **Detección de enfermedades (cáncer, por ejemplo):** un FN significa decirle a una persona enferma que está sana — puede ser fatal. Preferís algunos falsos positivos (decirle a alguien sano que podría estar enfermo, y luego descartarlo con más estudios) antes que dejar pasar un caso real.
- **Detección de fraudes:** dejar pasar una transacción fraudulenta (FN) es más costoso que marcar una transacción legítima para revisión extra (FP).
- **Ejemplo alternativo de Argentina:** "quiero celebrar cada juego que gana la selección" → ahí me interesa un Recall alto, no perderme festejar ningún partido ganado.

Cuando ambas importan por igual → F1-Score

15. EL PROBLEMA DE ACCURACY CON CLASES DESBALANCEADAS

Accuracy sola puede ser engañosa. Ejemplo clásico de examen:

Si el 98% de un dataset médico son personas sanas y solo el 2% tienen una enfermedad rara, un modelo que **siempre prediga "sano"** (sin haber aprendido nada) va a tener:

Accuracy = 98% ← ¡parece excelente!

Pero ese modelo tiene Recall = 0% para la clase enferma (nunca detecta ningún caso real de la enfermedad) — es completamente inútil para el propósito real. Por eso, con clases desbalanceadas, hay que mirar Precision, Recall y F1-Score, NO solo Accuracy.

16. MÉTRICAS DE REGRESIÓN (para cuando NO es clasificación)

Precision, Recall, Accuracy y F1 son SOLO para clasificación. Para regresión (predecir un valor numérico continuo) se usan otras métricas:

MAE (Mean Absolute Error)

Promedio de los errores absolutos entre predicción y valor real.

$$MAE = (1/n) * \sum |y_{real} - y_{predicho}|$$

- Penaliza todos los errores de forma lineal (por igual)
- No es tan sensible a errores grandes/outliers

MSE (Mean Squared Error)

Promedio de los errores al cuadrado.

$$\text{MSE} = (1/n) * \sum (y_{\text{real}} - y_{\text{predicho}})^2$$

- Penaliza mucho más los errores grandes (porque los eleva al cuadrado)
- Muy sensible a outliers
- Se usa principalmente para comparar entre modelos (al no tener raíz, la escala no es directamente interpretable, pero sirve para comparar)

RMSE (Root Mean Square Error)

Es la raíz cuadrada del MSE.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- Igual que MSE, penaliza más los errores grandes
- A diferencia de MSE, está en la MISMA escala/unidad que la variable original (por eso es más interpretable que MSE)

R² (Coeficiente de Determinación)

Mide qué proporción de la variabilidad de la variable objetivo es explicada por el modelo.

$$R^2 = 1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$$

Donde SSres es la suma de los cuadrados residuales (errores) y SStot es la variabilidad total de los datos.

- R² cercano a 1 → el modelo explica bien la variabilidad de los datos
- R² cercano a 0 → el modelo tiene poco poder explicativo
- R² negativo → el modelo es PEOR que simplemente predecir siempre la media
- No es sensible a la escala de los errores (a diferencia de MAE/MSE/RMSE)
- Puede ser engañoso con muchos features (tiende a sobreestimar el ajuste) → para esos casos se usa el R² ajustado

Tabla resumen:

Métrica	¿Penaliza más los errores grandes?	¿Misma escala que la variable?
MAE	No	Sí
MSE	Sí (al cuadrado)	No (al cuadrado)
RMSE	Sí	Sí
R ²	No aplica (es proporción)	No (es 0 a 1, o negativo)

17. EJERCICIOS REALES DE EXAMEN

Ejercicio (Recuperatorio 23-11-2023)

Se entrenaron dos modelos de clasificación para detectar personas sanas (clase 0) y personas enfermas (clase 1). Se evaluaron en el test y se obtuvieron las matrices de confusión con estos valores para la Clase 1:

Modelo A: TP=22, FN=16, FP=12, TN=13

Modelo B: TP=27, FN=11, FP=10, TN=15

a) F1-Score Modelo A:

$$\text{Recall}_A = 22 / (22+16) = 22/38 = 11/19 \approx 0,579$$

$$\text{Precision}_A = 22 / (22+12) = 22/34 = 11/17 \approx 0,647$$

$$\text{F1}_A = 2 * (0,647 * 0,579) / (0,647 + 0,579) \approx 0,611$$

b) F1-Score Modelo B:

$$\text{Recall}_B = 27 / (27+11) = 27/38 \approx 0,711$$

$$\text{Precision}_B = 27 / (27+10) = 27/37 \approx 0,730$$

$$\text{F1}_B = 2 * (0,730 * 0,711) / (0,730 + 0,711) \approx 0,720$$

c) ¿Qué modelo elegiría en términos de Accuracy?

$$\text{Accuracy}_A = (TN+TP) / (TN+TP+FP+FN) = (13+22) / 63 = 35/63 \approx 0,556$$

$$\text{Accuracy}_B = (TN+TP) / (TN+TP+FP+FN) = (15+27) / 63 = 42/63 \approx 0,667$$

El modelo B tiene mayor Accuracy (y también mayor F1-Score), por lo que se elegiría el Modelo B.

Ejercicio (Ejemplo de parcial)

Modelo A evaluado en test dio: Precision = 0,755, Recall = 0,744.

Modelo B evaluado en los mismos datos de test:

TP=49, TN=130, FP=41, FN=100

Calcular Accuracy del Modelo B:

$$\begin{aligned}\text{Accuracy}_B &= (TP+TN) / (TP+TN+FP+FN) = (49+130) / (49+130+41+100) \\ &= 179 / 320 \approx 0,559\end{aligned}$$

Con este dato de Accuracy (0,769 según la clave del examen — puede variar levemente según los valores completos de la matriz de A, que no estaban en el texto extraído), se compara contra el Accuracy del Modelo A y se elige el mayor.

Nota: este ejercicio muestra un patrón típico: te dan la matriz completa de un modelo y solo Precision/Recall del otro (o viceversa), obligándote a calcular la métrica faltante para poder comparar.

PREGUNTAS DE PRÁCTICA — PARTE 2 (Precision/Recall/F1)

Pregunta 1 (Cálculo)

Un modelo de clasificación binaria fue evaluado y se obtuvo la siguiente matriz de confusión:

	Predicted Pos	Predicted Neg
Actual Pos	45	15
Actual Neg	10	80

Calcule: a) Precision, b) Recall, c) Accuracy, d) F1-Score

Pregunta 2 (Desarrollo — conceptual)

Explique la diferencia entre Precision y Recall. Dé un ejemplo de un problema donde priorizaría Precision, y otro donde priorizaría Recall.

Pregunta 3 (V/F)

- a) Un modelo con Accuracy del 95% siempre es un buen modelo.
- b) F1-Score es la media armónica entre Precision y Recall.
- c) Un Falso Negativo es cuando el modelo predijo positivo pero el valor real era negativo.
- d) MAE penaliza los errores grandes más fuertemente que MSE.

Pregunta 4 (Desarrollo)

¿Por qué Accuracy puede ser una métrica engañosa cuando las clases están desbalanceadas? Dé un ejemplo concreto.

RESPUESTAS MODELO — PARTE 2

Respuesta 1

$$\begin{aligned}\text{Precision} &= TP/(TP+FP) = 45/(45+10) = 45/55 \approx 0,818 \\ \text{Recall} &= TP/(TP+FN) = 45/(45+15) = 45/60 = 0,75 \\ \text{Accuracy} &= (TP+TN)/(TP+TN+FP+FN) = (45+80)/150 = 125/150 \approx 0,833 \\ \text{F1} &= 2*(0,818*0,75)/(0,818+0,75) \approx 0,783\end{aligned}$$

Respuesta 2

Precision mide qué proporción de las predicciones positivas del modelo fueron correctas ($TP/(TP+FP)$). Recall mide qué proporción de los casos realmente positivos el modelo logró detectar ($TP/(TP+FN)$).

Ejemplo priorizando Precision: un filtro de spam. Si el modelo marca como spam un email importante (Falso Positivo), el usuario puede perder información crítica. Es preferible dejar pasar algo de spam real (Falso Negativo) antes que bloquear emails legítimos.

Ejemplo priorizando Recall: detección de una enfermedad grave. Un Falso Negativo (decirle a un paciente enfermo que está sano) puede tener consecuencias fatales. Es preferible generar algunos Falsos Positivos (estudios adicionales a personas sanas) antes que dejar pasar un caso real.

Respuesta 3

a) **FALSO.** Con clases desbalanceadas, un Accuracy alto puede ser engañoso. Por ejemplo, si el 95% de los datos pertenece a una clase, un modelo que siempre prediga esa clase tendría 95% de Accuracy sin haber aprendido nada.

b) **VERDADERO.** $F1 = 2*(Precision*Recall)/(Precision+Recall)$, que es la fórmula de la media armónica entre ambas métricas.

c) **FALSO.** Eso describe un Falso Positivo. Un Falso Negativo es cuando el modelo predijo NEGATIVO pero el valor real era POSITIVO.

d) **FALSO.** Es al revés: MSE (y RMSE) penalizan más los errores grandes porque los elevan al cuadrado. MAE trata a todos los errores linealmente, por igual, sin penalizar más los grandes.

Respuesta 4

Accuracy puede ser engañosa porque no diferencia entre los tipos de error (FP vs FN) ni considera la proporción de cada clase. Si una clase domina el dataset, un modelo que prediga siempre esa clase mayoritaria obtiene un Accuracy alto sin haber aprendido ningún patrón real.

Ejemplo: en un dataset médico donde el 98% de los pacientes están sanos y solo el 2% tiene una enfermedad, un modelo que prediga "sano" para todos los casos tendría 98% de Accuracy, pero 0% de Recall para detectar la enfermedad — sería completamente inútil para el objetivo real del problema. En estos casos hay que evaluar con Precision, Recall y F1-Score en vez de (o además de) Accuracy.